

Mini-projet 08 — La composition : la classe Rapport

Pourquoi ce mini-projet ?

Après un scan, ta boîte à outils doit **regrouper** tous les résultats : quelles cibles, quels ports ouverts, combien au total. C'est le rôle d'une classe **Rapport** qui **contient** une liste de résultats. On passe de « un objet = une donnée » à « un objet qui **agrège** d'autres objets ». Et on règle au passage **ton vieux réflexe** : une collection s'initialise dans `__init__`, pas ailleurs.

1. Composition vs héritage : « a un » vs « est un »

Deux façons de relier des classes, à ne pas confondre :

Relation	Question	Exemple
Héritage (MP07)	« est un » ?	un <code>PluginPing</code> EST UN <code>Plugin</code>
Composition (ici)	« a un / contient » ?	un <code>Rapport</code> A UNE liste de <code>Resultat</code>

Le test mental : « une voiture EST-ELLE un moteur ? » → non. « une voiture A-T-ELLE un moteur ? » → oui → **composition** (la voiture contient un moteur). On assemble des objets comme des Lego.

2. Un objet qui en contient un autre : Resultat

Un `Resultat` associe une `Cible` à un statut. Il **contient** un objet `Cible` — c'est de la composition simple :

```
class Resultat:
    def __init__(self, cible, statut):
        self.cible = cible          # self.cible EST un objet Cible (composition)
        self.statut = statut        # "ouvert" ou "ferme"

    def __repr__(self):
        return f"{self.cible.adresse()} [{self.statut}]"
        # on plonge DANS l'objet contenu : self.cible.adresse()
```

```
r = Resultat(Cible("10.0.0.5", 22), "ouvert")
print(r)                # 10.0.0.5:22 [ouvert]
print(r.cible.port)     # 22  <- on traverse : Resultat -> Cible -> port
```

Chaînage : `r.cible.adresse()` se lit « le `.cible` de `r`, puis son `.adresse()` ». Chaque point descend d'un cran dans l'assemblage d'objets.

3. ⚠ Une collection à l'intérieur : LE piège à connaître

Le `Rapport` contient une **liste** de résultats. Question cruciale : où et comment créer cette liste ?

✗ LA MAUVAISE FAÇON (deux variantes, deux bugs) :

```
# Variante A – liste en paramètre par défaut : le PIÈGE du défaut mutable (revu au MP02)
class Rapport:
    def __init__(self, resultats=[]):  # ✗ tous les Rapports partagent LA MÊME liste !
        self.resultats = resultats

# Variante B – obliger l'appelant à fournir la liste : inutile et fragile
class Rapport:
    def __init__(self, resultats):     # ✗ on doit passer [] à la main à chaque fois
        self.resultats = resultats
```

✓ LA BONNE FAÇON — créer la liste vide DANS le constructeur :

```
class Rapport:
    def __init__(self):
        self.resultats = [] # une liste NEUVE, propre à CHAQUE Rapport
```

Pourquoi c'est la règle : chaque `Rapport()` exécute `self.resultats = []` → il obtient **sa propre** liste vide. Deux rapports ne se marchent jamais dessus. C'est exactement ta règle « une collection s'initialise dans `__init__` , on ne la passe pas en paramètre ».

4. Les méthodes d'agrégation

Une fois la liste en place, le `Rapport` offre des méthodes pour la remplir et la résumer :

```
class Rapport:
    def __init__(self):
        self.resultats = []

    def ajouter(self, resultat):
        self.resultats.append(resultat)

    def nb_ouverts(self):
        return sum(1 for r in self.resultats if r.statut == "ouvert")
        # la technique de comptage du MP01 (bonus), appliquée aux objets
```

Tu réutilises tout : le `sum(1 for ... if ...)` vient du MP01, appliqué ici à une liste d'objets `Resultat` . Les briques s'empilent.

🎯 Ta mission

Objectif : deux classes en composition — `Resultat` (contient une `Cible`) et `Rapport` (contient une liste de `Resultat`).

Prépare : recopie ta classe `Cible` en haut du fichier.

- `class Resultat : __init__(self, cible, statut)` + un `__repr__` qui affiche `"ip:port [statut]"` .
- `class Rapport :`
 - `__init__(self)` → crée `self.resultats = []` (liste vide, **dans** le constructeur !)
 - `ajouter(self, resultat)` → ajoute un résultat à la liste ;
 - `nb_ouverts(self)` → renvoie le nombre de résultats dont le statut est `"ouvert"` ;
 - `resume(self)` → renvoie une chaîne `"X résultats, Y ouverts"` .

Tests à faire passer :

```
rapport = Rapport()
rapport.ajouter(Resultat(Cible("10.0.0.5", 22), "ouvert"))
rapport.ajouter(Resultat(Cible("10.0.0.5", 80), "ouvert"))
rapport.ajouter(Resultat(Cible("10.0.0.5", 3306), "ferme"))

print(rapport.resultats) # -> [10.0.0.5:22 [ouvert], 10.0.0.5:80 [ouvert], 10.0.0.5:3306 [ferme]]
print(rapport.nb_ouverts()) # -> 2
print(rapport.resume()) # -> 3 résultats, 2 ouverts

# LE test qui prouve que tu n'as pas le bug du défaut mutable :
r1 = Rapport()
r2 = Rapport()
r1.ajouter(Resultat(Cible("1.1.1.1", 22), "ouvert"))
print(len(r1.resultats), len(r2.resultats)) # -> 1 0 (r2 NE DOIT PAS avoir hérité du résultat de r1 !)
```

Bonus : ajoute une méthode `__len__(self)` qui renvoie `len(self.resultats)` — ça te permet d'écrire `len(rapport)` directement.

✓ Critères de réussite

- `rapport.resume()` renvoie `"3 résultats, 2 ouverts"` .
- Test anti-bug :** `r1` et `r2` ont des listes **indépendantes** (`1 0` , pas `1 1`).
- `__repr__` de `Resultat` plonge bien dans la `Cible` (`r.cible.adresse()`).

Si tu bloques — indices progressifs

Étape	Indice
Resultat.__repr__	<code>return f"{self.cible.adresse()} [{self.statut}]"</code> — tu traverses vers la Cible contenue.
Liste dans Rapport	<code>__init__(self):</code> avec juste <code>self.resultats = []</code> dedans. Pas de paramètre <code>resultats</code> .
nb_ouverts	<code>sum(1 for r in self.resultats if r.statut == "ouvert")</code> .
resume	Combine <code>len(self.resultats)</code> et <code>self.nb_ouverts()</code> dans une f-string.

Où ranger : `mini-projets/08-composition-rapport/rapport.py` .
Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ».